PRACTICAL WORK 3

ROS2 Vision-Based Hardware Control Using ESP32

Training details	Participant information
OS / ROS2	Ubuntu 24.04 / ROS2 Jazzy
Workspace / package	~/mindspark_ws / mindspark_robot
ROS_DOMAIN_ID	30 (same value in every terminal)
Name / Group	_____
Date / Instructor	_____

1. Aim and learning outcomes

Aim: Integrate existing ROS2 vision nodes with a decision node and an ESP32 serial control node to operate three external LED outputs based on the detected green ball position.

LO1 Implement vision-based LEFT/CENTER/RIGHT/NOT-DETECTED decisions using ROS2 topic messages.

LO2 Interface a ROS2 subscriber node with an ESP32 via USB serial communication.

LO3 Integrate and verify a multi-node camera-to-hardware workflow.

LO4 Create a Python ROS2 launch file and validate system operation using ROS2 CLI.

2. Prerequisites and equipment

Item	Specification / note
Existing PW2 package	camera_node.py and green_ball_node.py operational
Laptop	ROS2 Jazzy, OpenCV, cv_bridge, Python 3
Camera	USB webcam; image topic /camera/image_raw
Controller	ESP32 DevKit, connected at /dev/ttyUSB0
Outputs	Red, yellow and green LED modules; suitable wiring / common GND
Software	Arduino IDE (firmware), python3-serial, colcon

Baseline: camera image 320 × 240; decision output 5 Hz. Camera acquisition rate may vary; 30 FPS is a target rather than a prerequisite for PW3.

3. System design

```
USB Webcam → camera_node → /camera/image_raw
             green_ball_node → /green_ball
             vision_decision_node → /vision/decision
             esp32_control_node → USB Serial (/dev/ttyUSB0)
             ESP32 → RED / YELLOW / GREEN LED
```

Position / event	Decision	ESP32 output
LEFT (x = 0–105)	L	Red LED ON (GPIO25)
CENTER (x = 106–212)	C	Yellow LED ON (GPIO26)
RIGHT (x = 213–319)	R	Green LED ON (GPIO27)
No detection / timeout	N	All LEDs OFF

Note: These exact boundaries follow integer division in the provided Python code (320 // 3 = 106; 2 × 320 // 3 = 213).

4. Activity A — ESP32 firmware and hardware setup

A1. Wire the LED modules according to their electrical specifications. For active-HIGH modules: GPIO25 → RED input, GPIO26 → YELLOW input, GPIO27 → GREEN input, and common GND. Connect VCC only if required by the module and at the rated voltage.

LED Module	ESP32 GPIO	Function
Red LED	GPIO 25	LEFT
Yellow LED	GPIO 26	CENTER
Green LED	GPIO 27	RIGHT
GND	GND	Common ground

A2. Connect ESP32 via USB. Upload the following code using Arduino IDE (115200 baud).

```
const int redLED = 25;
const int yellowLED = 26;
const int greenLED = 27;

void allLEDsOff() {
  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, LOW);
}

void setup() {
  Serial.begin(115200);
```

```
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  allLEDsOff();
  Serial.println("PW3 ESP32 LED Controller Ready");
}

void loop() {
  if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == '\n' || command == '\r') return;
    allLEDsOff();
    switch (command) {
      case 'L': case 'l': digitalWrite(redLED, HIGH); break;
      case 'C': case 'c': digitalWrite(yellowLED, HIGH); break;
      case 'R': case 'r': digitalWrite(greenLED, HIGH); break;
      case 'N': case 'n': break;
      default: Serial.println("Invalid command"); break;
    }
    Serial.print("Command: ");
    Serial.println(command);
  }
}
```

A3. Using Arduino **Serial Monitor** (115200 baud), test L, C, R and N. Close Serial Monitor before ROS2 accesses the port.

Input	Expected Serial Output	Expected Hardware
L	LEFT - RED LED ON	Red ON
C	CENTER - YELLOW LED ON	Yellow ON
R	RIGHT - GREEN LED ON	Green ON
N	NONE - ALL LEDs OFF	All OFF

Command	Expected result	Actual / pass
L	Red only	_____
C	Yellow only	_____
R	Green only	_____
N	All OFF	_____

SCREENSHOT EVIDENCE | A — ESP32 firmware / wiring and LED test

Paste screenshot here (terminal, ROS2 CLI, camera view or hardware output).

5. Activity B — vision decision node

B1. Reuse camera_node.py and green_ball_node.py from PW2. The detection topic /green_ball is std_msgs/msg/String and follows the tested format DETECTED: x=150, y=120 or NOT DETECTED.

6. Test Serial Port from Ubuntu

Selepas berjaya dengan Arduino Serial Monitor, tutup Serial Monitor dahulu.

Buka Ubuntu Terminal:

```bash
ls /dev/ttyUSB*
```

Jika ESP32 dikesan sebagai `/dev/ttyUSB0`, semak permission:

```bash
ls -l /dev/ttyUSB0
```

Kemudian semak kumpulan pengguna:

```bash
groups
```

Jika `dialout` belum disenaraikan, jalankan:

```bash
sudo usermod -aG dialout $USER
```

B2. Create the decision node:

Node	Subscribe	Publish	Message Type
camera_node	—	/camera/image_raw	sensor_msgs/Image
green_ball_node	/camera/image_raw	/green_ball	std_msgs/String
vision_decision_node	/green_ball	/vision/decision	std_msgs/String

```
cd ~/mindspark_ws/src/mindspark_robot/mindspark_robot
code vision_decision_node.py
#!/usr/bin/env python3
import re
import time
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class VisionDecisionNode(Node):
    def __init__(self):
        super().__init__('vision_decision_node')
        self.frame_width = 320
        self.left_boundary = self.frame_width // 3
        self.right_boundary = (self.frame_width * 2) // 3
        self.current_decision = 'N'
        self.last_logged_decision = None
        self.last_message_time = time.monotonic()
        self.timeout_sec = 1.0
        self.subscription = self.create_subscription(
            String, '/green_ball', self.ball_callback, 10)
        self.publisher = self.create_publisher(
            String, '/vision/decision', 10)
        self.timer = self.create_timer(0.2, self.publish_current_decision)
        self.get_logger().info('Vision Decision Node Started (5 Hz)')

    def ball_callback(self, msg):
        self.last_message_time = time.monotonic()
        data = msg.data.strip()
        if data.startswith('NOT DETECTED'):
            self.current_decision = 'N'
            return
        if not data.startswith('DETECTED:'):
            self.current_decision = 'N'
            self.get_logger().warning(f'Unknown message: {data}')
            return
        match = re.search(r'\bx\s*=\s*(\d+)', data)
        if not match:
            self.current_decision = 'N'
            return
        x = int(match.group(1))
        if not 0 <= x < self.frame_width:
            self.current_decision = 'N'
        elif x < self.left_boundary:
            self.current_decision = 'L'
        elif x < self.right_boundary:
```

```
            self.current_decision = 'C'
        else:
            self.current_decision = 'R'

    def publish_current_decision(self):
        if time.monotonic() - self.last_message_time > self.timeout_sec:
            self.current_decision = 'N'
        msg = String()
        msg.data = self.current_decision
        self.publisher.publish(msg)
        if msg.data != self.last_logged_decision:
            self.get_logger().info(f'Vision Decision: {msg.data}')
            self.last_logged_decision = msg.data

def main(args=None):
    rclpy.init(args=args)
    node = VisionDecisionNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

B3. Register the executable in **setup.py** under console_scripts:

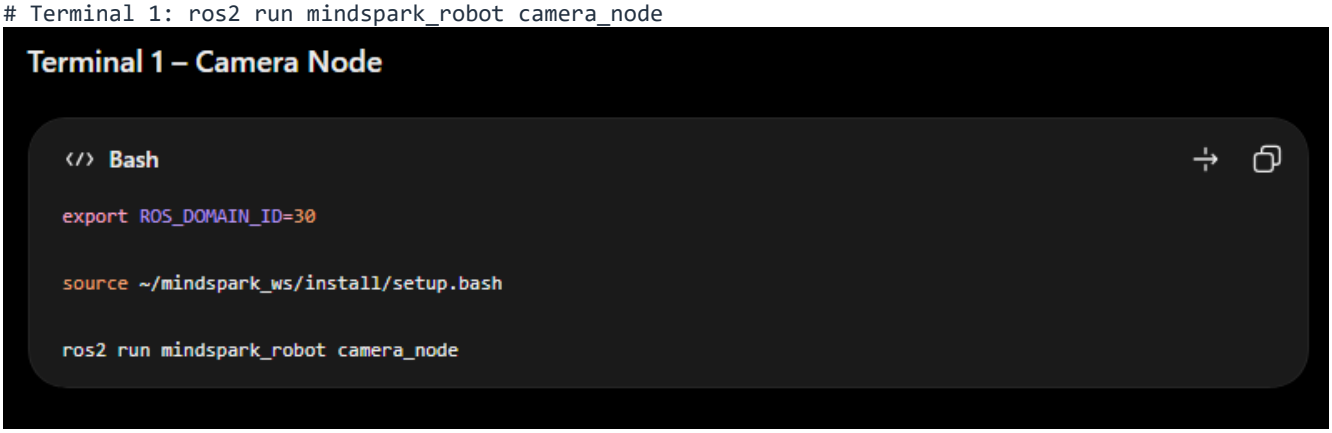
```
'vision_decision_node = mindspark_robot.vision_decision_node:main',
```

B4. Build and source:

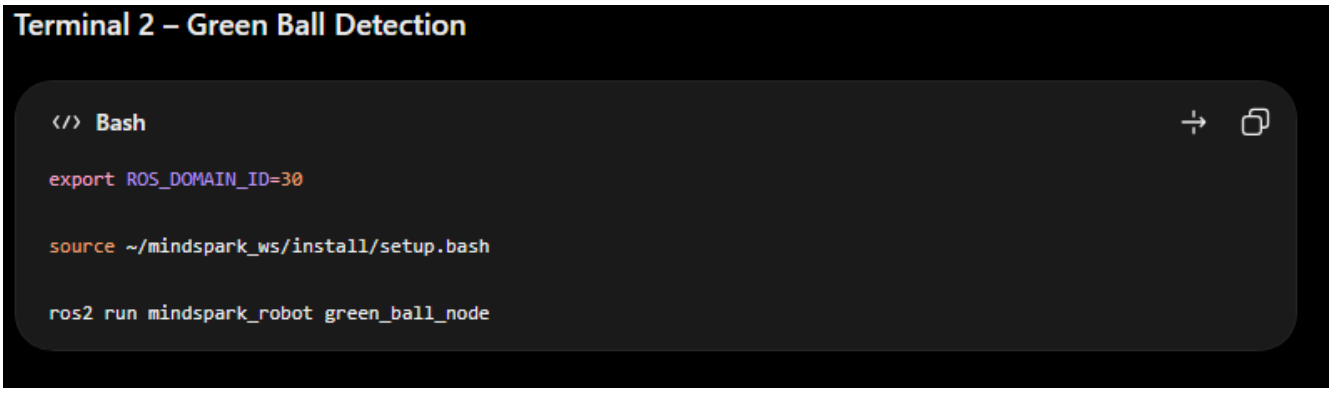
```
cd ~/mindspark_ws
colcon build --symlink-install --packages-select mindspark_robot
source install/setup.bash
```

B5. Open three terminals; set domain and source the workspace in EACH terminal. Run one node per terminal:

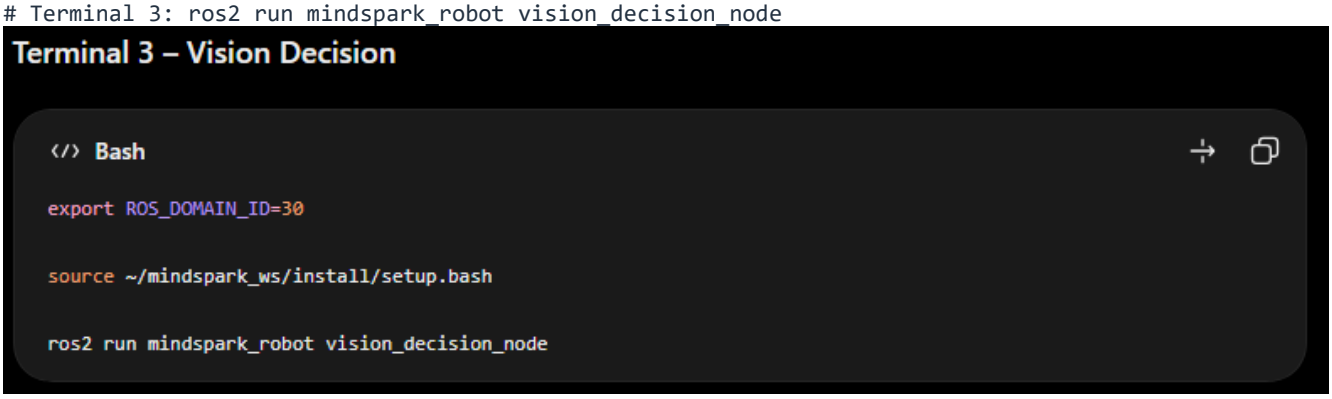
```
export ROS_DOMAIN_ID=30
source ~/mindspark_ws/install/setup.bash
# Terminal 1: ros2 run mindspark_robot camera_node
```




```
# Terminal 2: ros2 run mindspark_robot green_ball_node
```




```
# Terminal 3: ros2 run mindspark_robot vision_decision_node
```



Expected output:

Plain text

```
[INFO] Vision Decision Node Started
[INFO] Waiting for green ball detection...
```

Expected output:

Plain text

```
[INFO] Vision Decision Node Started
[INFO] Waiting for green ball detection...
```

Apabila green ball dikesan:

Plain text

```
[INFO] Vision Decision: L
```

Gerakkan bola ke tengah:

Plain text

```
[INFO] Vision Decision: C
```

Gerakkan ke kanan:

Plain text

```
[INFO] Vision Decision: R
```

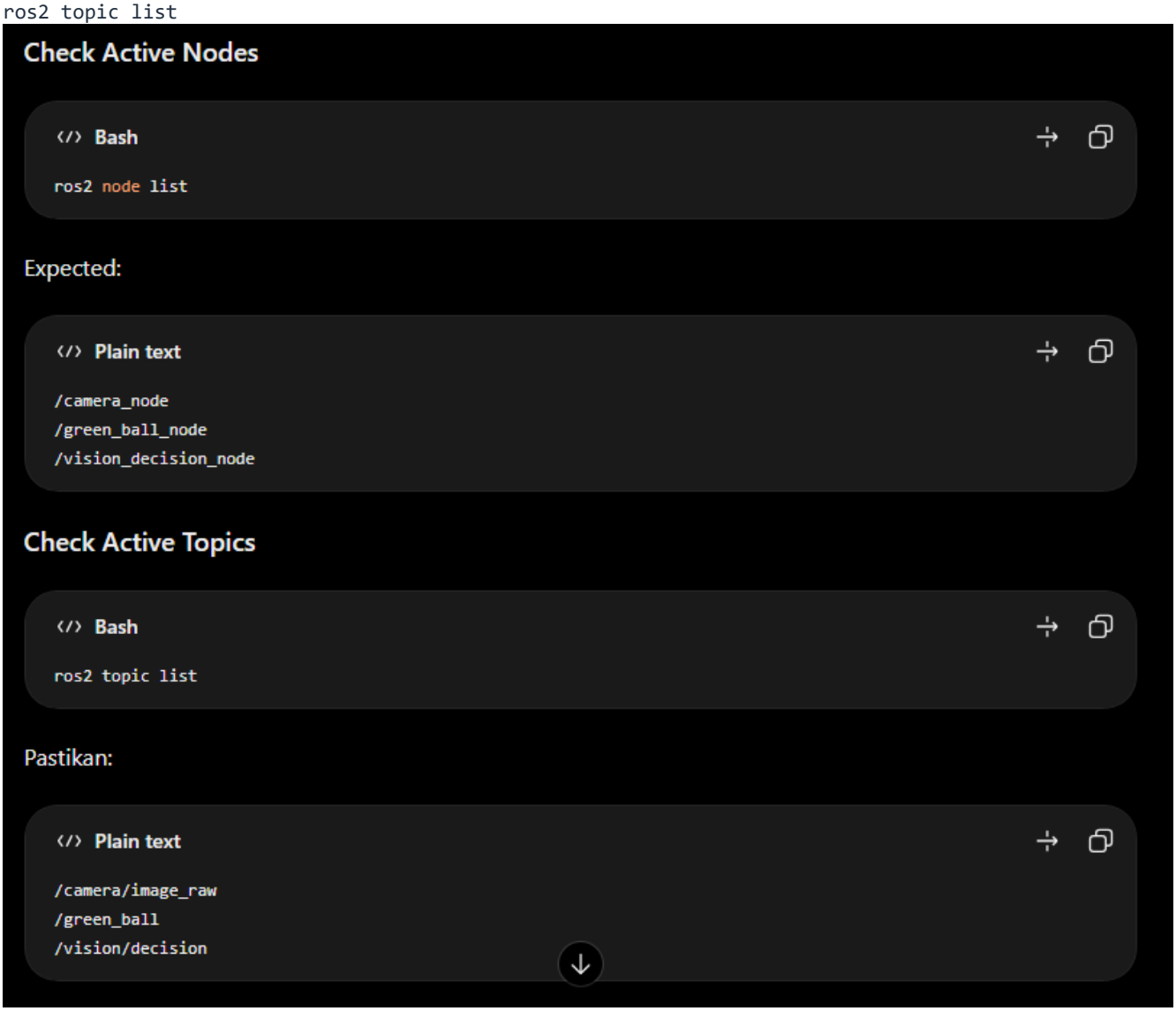
Keluarkan bola daripada kamera:

Plain text

```
[INFO] Vision Decision: N
```

B6. Verify active topics and messages in **another** terminal:

```
ros2 node list
ros2 topic list
```




```
ros2 topic info /green_ball -v
ros2 topic echo /vision/decision
ros2 topic hz /vision/decision
```

Expected: /vision/decision carries L/C/R/N; approximately 5 Hz; /green_ball has one detection publisher and one decision subscriber (additional CLI monitoring creates extra subscribers).

SCREENSHOT EVIDENCE | B — Vision decision echo, topic info and 5 Hz

Paste screenshot here (terminal, ROS2 CLI, camera view or hardware output).

6. Activity C — ROS2 serial control node

C1. Verify **/dev/ttyUSB0** is present and the current user has permission**. Ensure Arduino Serial Monitor is closed.**

Parameter	Value
Device	ESP32 DevKit
Serial Port	/dev/ttyUSB0
Baud Rate	115200
ROS2 Topic	/vision/decision
Message Type	std_msgs/msg/String
Commands	L, C, R, N
LED GPIO	Red: 25, Yellow: 26, Green: 27

```
ls -l /dev/ttyUSB0
groups
python3 -c "import serial; print(serial.__version__)"
# If needed: sudo apt install python3-serial
```

C2. Create the serial subscriber node:

```
code ~/mindspark_ws/src/mindspark_robot/mindspark_robot/esp32_control_node.py
#!/usr/bin/env python3
import time
import serial
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class ESP32ControlNode(Node):
    def __init__(self):
        super().__init__('esp32_control_node')
        self.serial_port = '/dev/ttyUSB0'
        self.baud_rate = 115200
        self.current_command = 'N'
```

```
        self.last_sent_command = None
        self.last_message_time = time.monotonic()
        self.timeout_sec = 2.0
        self.esp32 = serial.Serial(
            self.serial_port, self.baud_rate, timeout=1, write_timeout=1)
        time.sleep(2)  # ESP32 may reset when USB serial opens
        self.esp32.reset_input_buffer()
        self.subscription = self.create_subscription(
            String, '/vision/decision', self.decision_callback, 10)
        self.timer = self.create_timer(0.2, self.check_timeout)
        self.send_command('N')
        self.get_logger().info(f'ESP32 Connected: {self.serial_port}')

    def send_command(self, command):
        try:
            self.esp32.write(command.encode('utf-8'))
            self.esp32.flush()
            if command != self.last_sent_command:
                self.get_logger().info(f'ESP32 Command: {command}')
            self.last_sent_command = command
        except serial.SerialException as e:
            self.get_logger().error(f'Serial Error: {e}')

    def decision_callback(self, msg):
        command = msg.data.strip().upper()
        if command not in ('L', 'C', 'R', 'N'):
            self.get_logger().warning(f'Invalid command: {command}')
            return
        self.last_message_time = time.monotonic()
        self.current_command = command
        if command != self.last_sent_command:
            self.send_command(command)

    def check_timeout(self):
        if time.monotonic() - self.last_message_time > self.timeout_sec:
            self.current_command = 'N'
            if self.last_sent_command != 'N':
                self.get_logger().warning('ROS2 Communication Timeout')
                self.send_command('N')

    def shutdown(self):
        if self.esp32.is_open:
            try:
                self.esp32.write(b'N')
                self.esp32.flush()
            except serial.SerialException:
                pass
            self.esp32.close()

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = ESP32ControlNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    except serial.SerialException as e:
        print(f'ESP32 Connection Error: {e}')
```

```
    finally:
        if node is not None:
            node.shutdown()
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

C3. Register in **setup.py**, rebuild and source:

```
'esp32_control_node = mindspark_robot.esp32_control_node:main',
cd ~/mindspark_ws
colcon build --symlink-install --packages-select mindspark_robot
source install/setup.bash
```

C4. Manual topic-to-LED test: run esp32_control_node in Terminal 1; in Terminal 2 publish each command. Since the control node times out after 2 seconds, use continuous publishing for sustained LED tests.

```
ros2 run mindspark_robot esp32_control_node
# In a second terminal (same domain and sourced workspace):
ros2 topic pub --once /vision/decision std_msgs/msg/String "{data: L}"
ros2 topic pub --once /vision/decision std_msgs/msg/String "{data: C}"
ros2 topic pub --once /vision/decision std_msgs/msg/String "{data: R}"
ros2 topic pub --once /vision/decision std_msgs/msg/String "{data: N}"
# Continuous test (Ctrl+C to stop):
ros2 topic pub -r 5 /vision/decision std_msgs/msg/String "{data: L}"
```

C5. Verify subscriber:

```
ros2 node info /esp32_control_node
ros2 topic info /vision/decision -v
```

SCREENSHOT EVIDENCE | C — ROS2 manual commands and ESP32 LED reaction

Paste screenshot here (terminal, ROS2 CLI, camera view or hardware output).

7. Activity D — complete four-node integration

D1. Stop manual topic publisher and any duplicate nodes. Keep ESP32 connected. Start the four ROS2 nodes in separate terminals; set ROS_DOMAIN_ID=30 and source the workspace in each terminal.

Terminal	Command
1	ros2 run mindspark_robot camera_node
2	ros2 run mindspark_robot green_ball_node
3	ros2 run mindspark_robot vision_decision_node
4	ros2 run mindspark_robot esp32_control_node

D2. Check that the ROS2 graph contains the expected nodes and topics:

```
ros2 node list
ros2 topic list
ros2 topic info /vision/decision -v
ros2 topic hz /vision/decision
```

Test	Expected LED	Pass / notes
Green ball LEFT	RED	_____
Green ball CENTER	YELLOW	_____
Green ball RIGHT	GREEN	_____
Green ball removed	ALL OFF	_____
Green ball remains still	Selected LED stays ON	_____
Camera publisher stopped	ALL OFF after detection timeout	_____

SCREENSHOT EVIDENCE | D — Complete camera-to-ESP32 integration

Paste screenshot here (terminal, ROS2 CLI, camera view or hardware output).

8. Activity E — one-command ROS2 launch

Nodes to Launch

No.	Node	Executable
1	Camera	`camera_node`
2	Green Ball Detection	`green_ball_node`
3	Vision Decision	`vision_decision_node`
4	ESP32 Control	`esp32_control_node`

E1. **Create launch directory** and **launch file** at the package root:

```
cd ~/mindspark_ws/src/mindspark_robot
mkdir -p launch

code launch/pw3_hardware.launch.py

#!/usr/bin/env python3
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(package='mindspark_robot', executable='camera_node',
             name='camera_node', output='screen'),
        Node(package='mindspark_robot', executable='green_ball_node',
             name='green_ball_node', output='screen'),
        Node(package='mindspark_robot', executable='vision_decision_node',
             name='vision_decision_node', output='screen'),
        Node(package='mindspark_robot', executable='esp32_control_node',
             name='esp32_control_node', output='screen'),
    ])
```

E2. Configure ROS2 Package for Launch File Installation

Open the setup.py file and replace its contents with the following code. This configuration registers all ROS2 nodes and installs the launch file required for PW3.

code ~/mindspark_ws/src/mindspark_robot/setup.py

Full Code – setup.py

```
#!/usr/bin/env python3
from setuptools import find_packages, setup
import os
from glob import glob

package_name = 'mindspark_robot'

setup(
    name=package_name,
    version='0.0.0',

    packages=find_packages(exclude=['test']),

    # Install package resources and launch files
    data_files=[
        (
            'share/ament_index/resource_index/packages',
            ['resource/' + package_name]
        ),

        (
            'share/' + package_name,
            ['package.xml']
        ),

        (
            os.path.join('share', package_name, 'launch'),
            glob('launch/*.launch.py')
        ),
    ],

    install_requires=['setuptools'],
    zip_safe=True,
```

```
    maintainer='ilmi',
    maintainer_email='ilmiariffin@gmail.com',

    description='MindSpark ROS2 Training Package',
    license='Apache-2.0',

    tests_require=['pytest'],

    # Register ROS2 Python executables
    entry_points={
        'console_scripts': [

            # PW1 - Basic ROS2 Node
            'hello_node = mindspark_robot.hello_node:main',

            # PW2 - Camera and Ball Detection
            'camera_node = mindspark_robot.camera_node:main',
            'red_ball_node = mindspark_robot.red_ball_node:main',
            'green_ball_node = mindspark_robot.green_ball_node:main',

            # PW3 - Vision Decision and ESP32 Control
            'vision_decision_node = mindspark_robot.vision_decision_node:main',
            'esp32_control_node = mindspark_robot.esp32_control_node:main',

        ],
    },
)
```

E3. Build and Verify Launch File Installation:

```
cd ~/mindspark_ws
colcon build --symlink-install --packages-select mindspark_robot
source install/setup.bash
ls install/mindspark_robot/share/mindspark_robot/launch/
```

E4. Launch all nodes with one command:

```
export ROS_DOMAIN_ID=30
ros2 launch mindspark_robot pw3_hardware.launch.py
```

E5. From a second terminal, verify nodes, topics, topic details and frequency:

```
export ROS_DOMAIN_ID=30
source ~/mindspark_ws/install/setup.bash
ros2 node list
ros2 topic list
ros2 topic info /vision/decision -v
ros2 topic hz /vision/decision
ros2 topic hz /camera/image_raw
ros2 topic hz /green_ball
```

Expected: four application nodes, /camera/image_raw, /green_ball, /vision/decision; /vision/decision approximately 5 Hz. Camera and detection publishing rates depend on runtime camera performance.

SCREENSHOT EVIDENCE | E — Successful ros2 launch and four running nodes

Paste screenshot here (terminal, ROS2 CLI, camera view or hardware output).

9. Final verification record

Item to verify	Evidence / measured result
ESP32 serial port	/dev/ttyUSB0 | Baud 115200
ROS domain	30
/green_ball type	std_msgs/msg/String
/vision/decision type	std_msgs/msg/String
Decision publishing rate	_____ Hz (target 5 Hz)
Camera publishing rate	_____ Hz (record measured value)
Green detection publishing rate	_____ Hz (record measured value)
Node count	_____ active application nodes
LEFT / CENTER / RIGHT / NONE	Pass: _____ _____ _____ _____
Launch file run	Pass / fail: _____

10. Reflection and review questions

1. Explain why the system uses a separate vision_decision_node instead of controlling the ESP32 directly from green_ball_node.

_____

2. What is the difference between camera publishing frequency and vision decision publishing frequency?

_____

3. Why must ROS_DOMAIN_ID be consistent across the terminals?

_____

4. What happens after loss of /green_ball messages for more than one second?

_____

5. What is the benefit of using a launch file instead of running four separate ros2 run commands?

_____

11. Completion checklist

☐ ESP32 LED command tests completed

☐ Vision position decision L/C/R/N verified

☐ ROS2 serial command test completed

☐ Four-node live integration demonstrated

☐ Launch file executed and verified

☐ Required screenshots and measured rates recorded

Instructor verification: _____ Date: _____

Operational note: The ROS2 serial node times out to N when decision messages stop, but the tested ESP32 firmware has no independent watchdog for abrupt laptop/USB failure. This limitation is acceptable for this supervised LED-training demonstration.